

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct:

I Y.Naga Tharuni/192372338 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Naga Tharuni*

Annexure A

CONSTRAINT-BASED PROBLEM SOLVING

1. Intrusion Detection Deployment under Real-Time Constraints

A financial organization plans to deploy an Intrusion Detection System (IDS) with the following constraints:

- Must analyze traffic in real-time with latency ≤ 5 ms per packet
- Detection accuracy $\geq 96\%$
- False positive rate $\leq 1.5\%$
- Network traffic load: 150,000 packets per second
- Limited CPU resources (shared with other services)

Task:

Design an optimized IDS configuration (signature-based, anomaly-based, or hybrid). Analyze trade-offs between detection accuracy and system performance, and justify how your solution meets all constraints.

Introduction

A financial organization requires an Intrusion Detection System (IDS) that can analyze 150,000 packets per second while maintaining a latency of ≤ 5 ms per packet, detection accuracy of $\geq 96\%$, and a false positive rate of $\leq 1.5\%$. Since CPU resources are limited and shared with other services, a carefully optimized hybrid IDS is the most suitable solution.

Proposed IDS Configuration

A hybrid IDS combines signature-based detection with lightweight anomaly-based detection.

- **Signature-based IDS:** Detects known attacks using a database of verified attack signatures.
- **Anomaly-based IDS:** Detects unusual traffic patterns that may represent unknown attacks.
- **Traffic filtering:** Normal and low-risk traffic is processed using lightweight rules, while suspicious traffic receives deeper inspection.
- **Priority-based processing:** Critical protocols and high-risk traffic are analyzed first.

- **Signature optimization:** Only relevant and frequently used signatures are loaded into memory.
- **Hardware/software acceleration:** Packet inspection and rule matching should be optimized using efficient data structures and parallel processing where available.

Processing Strategy

The incoming traffic can be processed in three stages:

Packet → Fast Signature Filtering → Lightweight Anomaly Analysis → Alert/Allow

Known malicious packets can be rejected quickly through signature matching. Only packets that are unusual or suspicious are sent for more detailed anomaly analysis. This reduces unnecessary CPU consumption.

Trade-offs

A purely signature-based IDS provides high speed and low CPU usage but cannot effectively detect new or zero-day attacks. A purely anomaly-based IDS can detect unknown attacks but generally requires more processing and can generate more false positives.

The hybrid approach provides a balance:

Approach	Accuracy	Performance	Unknown Attack Detection
Signature-based	High for known attacks	Very high	Low
Anomaly-based	Variable	Lower	High
Hybrid	High	High	Moderate to High

Justification

The proposed configuration can meet the constraints by using fast signature matching for most traffic and applying anomaly detection only to suspicious packets. The target design should be validated through load testing at 150,000 packets/second and tuned until measured latency remains at or below 5 ms, accuracy is at least 96%, and false positives remain at or below 1.5%.

Therefore, a hybrid IDS with optimized signatures, selective anomaly analysis, traffic prioritization, and efficient packet processing is the most appropriate solution.

2. Worm Containment Strategy under Network Availability Constraints

An enterprise network faces rapid worm propagation with the following constraints:

- Network downtime must be $\leq 1\%$
- Detection and containment must occur within 2 minutes
- Systems are distributed across multiple locations
- Limited centralized control

Task:

Design a containment strategy using firewalls, IDS/IPS, and segmentation. Evaluate trade-offs between rapid isolation and business continuity, and justify how your approach minimizes spread while maintaining network availability.

Introduction

Rapidly spreading worms can infect large numbers of systems within minutes. The organization must contain the worm within 2 minutes while keeping network downtime below 1%. Since systems are distributed across multiple locations and centralized control is limited, containment must be decentralized and automated.

Proposed Containment Strategy

The strategy should use IDS/IPS, firewalls, and network segmentation together.

1. IDS/IPS Detection

1. Deploy IDS/IPS sensors at major network segments.
2. Detect unusual scanning, repeated connection attempts, and known worm signatures.
3. Generate alerts immediately when worm-like behavior is detected.

2. Firewall Blocking

1. Automatically block malicious source addresses and suspicious ports.
2. Restrict unnecessary east-west traffic between internal systems.
3. Apply temporary blocking rules during an incident.

3. Network Segmentation

1. Divide the enterprise into separate security zones.
2. Critical servers, user systems, databases, and branch networks should be placed in separate segments.

3. Use VLANs and firewall policies to control communication between segments.

4. Local Automated Response

1. Each location should have the ability to isolate an infected segment without waiting for centralized approval.
2. Only the affected segment should initially be isolated.

5. Recovery and Monitoring

1. Continuously monitor isolated systems.
2. Remove temporary blocking rules after the threat is eliminated.
3. Restore communication gradually after verification.

Containment Flow

Worm Detected → Identify Affected Segment → Block Malicious Traffic → Isolate Segment → Scan/Remove Worm → Verify → Restore Connectivity

Trade-offs

Completely shutting down the network would provide strong containment but could seriously affect business operations. On the other hand, allowing all systems to remain connected would maintain availability but allow the worm to spread.

Therefore, selective isolation provides the best balance.

Strategy	Security	Availability
Complete shutdown	Very High	Very Low
No isolation	Very Low	Very High
Selective segmentation	High	High

Justification

The proposed strategy limits isolation to affected network segments rather than shutting down the complete enterprise. Automated IDS/IPS responses allow detection and containment within the required 2-minute window, while segmentation prevents the infection from spreading between locations.

The design should be tested to ensure that the actual measured containment time is ≤ 2 minutes and network downtime remains $\leq 1\%$.

3. Firewall Rule Optimization under Performance Constraints

A company firewall currently uses 10,000+ rules, causing performance degradation. Constraints include:

- Throughput requirement: ≥ 800 Mbps
- Rule processing time must be minimized
- Must maintain strict access control policies
- No additional hardware allowed

Task:

Analyze inefficiencies in rule processing. Propose optimization techniques (rule aggregation, prioritization, redundancy elimination). Justify how your optimized rule set improves performance while maintaining security.

Introduction

A firewall containing more than 10,000 rules may experience performance degradation because packets may need to be compared against a large number of rules. The objective is to maintain strict access control while achieving a throughput of at least 800 Mbps.

Inefficiencies in the Existing Rule Set

Common problems include:

- Duplicate rules
- Redundant rules
- Shadowed rules
- Rules that are never matched
- Incorrect rule ordering
- Overlapping IP ranges
- Repeated port definitions
- Broad rules placed before specific security rules

Proposed Optimization Techniques

1. Rule Aggregation

Combine rules that have the same action and similar source, destination, and service requirements.

For example, multiple individual network addresses can be represented using an appropriate subnet when the security policy permits it.

2. Rule Prioritization

Frequently matched rules should be placed earlier in the rule set. High-risk blocking rules should also be positioned appropriately so that malicious traffic can be rejected quickly.

3. Redundancy Elimination

Duplicate and unnecessary rules should be identified and removed.

4. Shadow Rule Removal

A rule that can never be reached because an earlier rule already matches the same traffic should be removed or redesigned.

5. Object Groups

Instead of repeatedly specifying individual IP addresses and ports, use address groups and service groups where supported.

6. Default-Deny Policy

Use a clear default-deny rule at the end of the rule set. Only explicitly authorized traffic should be allowed.

Optimized Processing Structure

High-priority security rules → Frequently matched rules → Specific application rules → General permitted traffic → Default Deny

Trade-offs

Aggressive rule aggregation can improve performance but may accidentally allow traffic that should remain restricted. Therefore, rules should only be combined when their security conditions and actions are equivalent.

Similarly, placing frequently matched rules first improves performance, but security priority must remain more important than frequency.

Justification

Removing duplicates, eliminating shadowed rules, aggregating equivalent rules, and prioritizing frequently matched traffic significantly reduces the number of comparisons required for common packets.

The optimized firewall should then be load-tested to verify that it maintains ≥ 800 Mbps throughput while all mandatory access-control policies remain intact.

Therefore, rule aggregation, prioritization, redundancy elimination, and systematic rule auditing provide an effective method for improving firewall performance without adding hardware.

4. IPS Deployment under Resource and Accuracy Constraints

An organization wants to deploy an Intrusion Prevention System (IPS) with the following constraints:

- Must block $\geq 90\%$ of known attacks
- False positive rate $\leq 3\%$
- Available memory for signatures: 1 GB
- Network speed: 500 Mbps

Task:

Design an IPS configuration strategy (signature selection, traffic filtering, adaptive updates). Evaluate trade-offs between resource usage and detection capability, and justify how your solution satisfies all constraints.

Introduction

The organization requires an Intrusion Prevention System (IPS) capable of blocking at least 90% of known attacks, maintaining a false positive rate of $\leq 3\%$, operating within 1 GB of signature memory, and handling 500 Mbps network traffic.

Proposed IPS Configuration

1. Select High-Value Signatures

Instead of loading every available signature, select signatures based on:

- Current threat intelligence
- Frequently exploited vulnerabilities

- Operating systems and applications used by the organization
- High-severity attacks
- Known network attack patterns

This keeps the signature database within the 1 GB memory limit.

2. Traffic Filtering

Traffic should first pass through lightweight filtering.

Low-risk or trusted traffic can be processed using basic rules, while suspicious traffic receives deeper inspection.

3. Signature Prioritization

Signatures should be divided into:

- Critical
- High
- Medium
- Low

Critical and high-severity signatures receive priority.

4. Adaptive Updates

The IPS should regularly update its signature database. Old or irrelevant signatures can be disabled when they no longer provide useful protection.

5. False Positive Control

IPS rules should be tuned using actual network traffic. Repeated false positives should be investigated and corrected instead of simply disabling security controls.

Processing Model

Incoming Traffic → Basic Filtering → Signature Matching → Threat Verification
→ Block/Allow → Logging

Trade-offs

Loading more signatures can increase detection capability but consumes memory and processing resources. Loading fewer signatures improves performance but may reduce detection coverage.

Therefore, the IPS should focus on high-value signatures rather than attempting to load every available signature.

Configuration	Memory Usage	Detection Coverage	Processing Cost
All signatures	Very High	Very High	High
Limited signatures	Low	Lower	Low
Risk-based signatures	Controlled	High	Moderate

Justification

A risk-based signature strategy allows the IPS to remain within the 1 GB memory constraint while concentrating resources on the attacks most relevant to the organization. Traffic filtering and signature prioritization reduce unnecessary inspection.

The configuration should be validated using representative attack traffic to demonstrate $\geq 90\%$ known-attack blocking, $\leq 3\%$ false positives, and stable operation at 500 Mbps.

5. Malware Detection using Behavioral Analysis under System Constraints

A company plans to implement behavioral-based malware detection with the following constraints:

- Endpoint CPU usage must remain $\leq 15\%$
- Detection must occur within 10 seconds of execution
- Must detect unknown (zero-day) threats
- Limited storage for logs and analysis

Task:

Design a lightweight behavioral detection system. Analyze trade-offs between detection speed, accuracy, and resource consumption, and justify how your approach effectively identifies malware within the given constraints.

Introduction

Traditional signature-based antivirus systems may fail to detect unknown or zero-day malware. Therefore, the organization requires a lightweight behavioral detection system that detects suspicious behavior within 10 seconds, keeps endpoint CPU usage at $\leq 15\%$, and operates with limited storage.

Proposed Lightweight Behavioral Detection System

The system should monitor only important behavioral indicators rather than continuously analyzing every system event in detail.

Important indicators include:

- Unexpected process creation
- Suspicious child processes
- Unusual file modifications
- Unauthorized persistence mechanisms
- Abnormal network connections
- Rapid file encryption or modification
- Attempts to disable security software
- Unusual privilege changes

Lightweight Monitoring

A small endpoint agent can collect only security-relevant events. Instead of storing complete logs indefinitely, it can maintain a short rolling buffer.

For example:

Process Execution → Behavior Monitoring → Feature Extraction → Risk Score → Decision

Each behavior receives a risk score. Several suspicious behaviors occurring together increase the overall score.

Detection Strategy

The system can use a lightweight behavioral model or rule-based scoring system.

For example:

- Normal process behavior → Low risk
- Suspicious network connection → Medium risk
- Rapid modification of many files → High risk
- Persistence + suspicious process + abnormal network activity → Critical risk

When the risk score crosses a threshold, the endpoint can temporarily isolate the process or endpoint and send a compact alert to the security system.

Storage Optimization

Because storage is limited:

- Store security-relevant events only.
- Use a rolling log buffer.
- Compress older records.
- Store summaries instead of complete event streams.
- Retain detailed logs only for confirmed incidents.

Trade-offs

Approach	Detection Speed	CPU Usage	Zero-Day Detection
Full behavioral analysis	High	High	High
Signature-based detection	Very High	Low	Low
Lightweight behavioral analysis	High	Low to Moderate	High

A very detailed behavioral analysis could improve detection accuracy but may exceed the 15% CPU limit. Therefore, selective monitoring is preferred.

Justification

The proposed system focuses on high-value behavioral indicators and uses a risk-scoring mechanism instead of analyzing every system event. This reduces CPU and storage consumption while still allowing unknown malware to be detected through its behavior.

The system should be tested to verify that endpoint CPU usage remains $\leq 15\%$, suspicious executions are identified within 10 seconds, and sufficient behavioral indicators are monitored to detect previously unknown threats.

Conclusion

A lightweight behavioral detection system with selective event monitoring, risk scoring, rolling logs, and rapid response provides the best balance between detection capability, speed, CPU usage, and storage requirements.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and	Identifies most	Partial understanding	Fails to identify

		identifies all relevant constraints accurately	constraints with minor gaps	ng; misses key constraints	constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided